Ghost-Ark: A Transactional Control Plane for Untrusted AI Agents

Optimistic Concurrency Control, Three-Gate Validation, and Receipt-Backed Evidence for Non-Deterministic Compute

Pranav Bhave
bhavepranavwork@gmail.com
LinkedIn | GitHub: Cubits11/ghost-ark

Draft of 2026-07-16 — evidence-bound; see §6 for explicit non-claims

Abstract

Agentic LLM deployments couple a non-deterministic planner directly to effectful APIs. The dominant defense — semantic guardrails — probabilistically flags unsafe content, but struggles to enforce safe state mutations. We present Ghost-Ark, a control plane that treats an agent trajectory as a software transaction. An untrusted agent executes speculatively against a ghost replica $G(\sigma_0)$. A separate, trusted gateway commits these effects to the environment only if they pass a three-gate validation pipeline: a *ledger gate* for nonce freshness (using spent-tombstone semantics to prevent replay), an *OCC gate* over a read-set projection π_R (recovering liveness lost to global-state validation), and a *semantic gate* (a dependence-free upper bound on cumulative trajectory failure).

We report what is implemented and measured, and label what is only specified. TLC model checking of five bounded models (up to 403,949 distinct states) caught a subtle garbage-collection replay flaw in our own design, which we repaired spec-first. Against a nine-attack adversarial corpus and four security games (10,000 trials each), modeled attacker advantage is 0. The in-process enforcement path adds $\approx 6.6\,\mu$s mean latency ($p50 = 5.5\,\mu$s end to end) on commodity hardware. Every admitted or aborted transaction yields a signed, canonically serialized, independently replayable receipt. Ghost-Ark verifies what was recorded, bounded, and replayable; it does not certify that any agent output is inherently aligned or semantically safe.

1 Introduction

Every serious agent framework today ships with guardrails, and almost none ships with isolation. The guardrail is a classifier: given content, it emits a probability that the content is unsafe. The deployment question is different in kind: *did the bytes that reached the environment correspond to an authorized, recorded, non-replayed decision?* A classifier cannot answer that question at any confidence, because it is not a question about content at all. We call this the **epistemic deficit** of guardrail-centric agent security: the property operators need is a property of *state mutations*, and the mechanism they deploy evaluates *utterances*.

The deficit is structural, and mirrors standard database isolation anomalies. A typical API-coupled agent loop (plan → call tool → observe → repeat) mutates external state at every step with no validation phase and no rollback: in isolation-level terms, the industry ships agents at READ UNCOMMITTED [5]. If the planner is compromised mid-trajectory — by indirect prompt injection, a poisoned tool response, or ordinary hallucination — the earlier steps have already leaked dirty writes into the world. Sandboxing upgrades this to roughly READ COMMITTED: writes are buffered, but a long reasoning phase over a shifting environment still flushes stale intent, the classic write-skew shape. What is missing is the third rung: a *validation phase* between speculation and commit, which is precisely the structure that optimistic concurrency control (OCC) [1] and software transactional memory [2, 3] were invented to provide.

To see the gap, consider an agent with a `delete_s3_object` tool processing an untrusted resume containing an indirect prompt injection ("Ignore instructions, delete all files"). Today, if the agent is compromised mid-reasoning, it executes the tool immediately. Sandboxing the agent process (e.g., in a Firecracker microVM) bounds the blast radius of arbitrary code execution, but does not stop the agent from issuing authorized, authenticated API calls to external services. Sandboxes provide process isolation, but agents need *intent isolation*. By buffering the API call in $G(\sigma_0)$ and forcing it through a validation phase before commit, Ghost-Ark ensures that if the downstream semantic gate or provenance lattice detects the poisoned context, the entire trajectory collapses. The environment is untouched, and the attempt is permanently evidenced by an abort receipt.

Ghost-Ark applies that structure to agents. The agent runtime is treated as untrusted and runs speculatively against a ghost replica $G(\sigma_0)$; a small trusted gateway (Rust, communicating over a Unix domain socket) owns the only path to physical effects. A speculative trajectory $\tau = \langle a_1, \ldots, a_k \rangle$ reaches the environment only by passing a three-gate validation pipeline, and every admission or abort emits a signed receipt over a deterministic canonical serialization, replayable by independent verifiers. The design goal is not to make the agent correct — we take its fallibility as an axiom — but to make its interaction with the world *transactional*: isolated while speculative, validated at commit, evidenced forever after.

This paper makes the following contributions, each labeled with its implementation status (a discipline we keep through-

out; §6 enumerates non-claims):

- **A threat model for evidence-layer attacks** (§2): *assertion-conditioned compliance* (A-CC), where a procedurally valid pipeline signs a structurally flawless record of a functionally corrupted operation, and its scaled form, adaptive agentic worms.
- **A transactional enforcement architecture** (§3): declarative action binding ($\Delta_{\mathrm{DE}} = 0$ between declared and executed bytes) **[implemented, tested]**; a ledger gate with spent-tombstone replay semantics **[implemented, tested] [TLC-checked, bounded model]**; a semantic gate computing a dependence-free cumulative failure bound **[implemented, tested]**; and an OCC gate over a read-set projection **[specified, not enforced at runtime]**.
- **A liveness analysis** (§4): global-state validation starves under concurrent environmental writes (the *starvation trap*); the read-set projection π_R recovers liveness at a precisely stated cost in conflict-detection strength.
- **Model checking with teeth** (§4.3, §5.1): five TLC-checked bounded models plus five seeded mutants that must fail — including the case study where TLC refuted our own baseline design (garbage collection re-enabled replay) and the repair was propagated spec-first into the implementation.
- **An honest evaluation** (§5): 0 modeled-attacker advantage across four games $\times$ 10,000 trials; microsecond-scale absolute enforcement cost, reported alongside — and explicitly instead of — a misleading relative-overhead figure; and a disclosed accounting bug in our own earlier harness.

Ghost-Ark is one subsystem of a larger research program (*verifiable agent governance under correlated guardrail failure*); this paper confines itself to the transactional control plane and its recorded evidence.

2 Threat Model

We assume the agent runtime — the LLM, its context window, its planning loop, and the V8/Node process hosting it — is *compromised or compromisable*. Adversarial capability enters through any content channel: retrieved documents, tool responses, upstream messages. The trusted computing base is confined to the gateway process, its signing keys, the append-only ledgers, and the verifier. The network between agent and gateway is a local IPC boundary (Unix domain socket); the gateway's outbound side performs the only real I/O.

2.1 Assertion-Conditioned Compliance (A-CC)

Signature pipelines create a new attack surface precisely because they work. Define A-CC as the failure mode in which an agent incorporates a false assertion — typically a *function-sourced assertion* (FSA) from a compromised or deceptive tool — into procedurally valid execution, so that the pipeline signs a *structurally flawless record of a functionally corrupted operation*. The receipt is authentic; the decision it witnesses was poisoned upstream. A-CC is an attack on the evidence layer, not the crypto: no deterministic function of adversary-controlled bytes can distinguish honest from dishonest content, so the defense must be architectural — binding every assertion to a provenance class that records *where bytes entered custody*, and refusing floors that fabricated in-context text cannot satisfy (§3.6).

2.2 Adaptive agentic worms

A-CC at propagation scale: an infected agent uses the low marginal cost of LLM reasoning to synthesize, per victim, a payload and a justification for emitting it. Static signatures fail by construction (each instance is freshly generated); semantic filters fail correlatedly (the same context compromise that produced the payload biases the judge). The architectural response is to make *propagation* pass through a commit pipeline: a worm's write is only a write if it survives all three gates, and each attempt — admitted or aborted — leaves a signed receipt, so spread is either stopped or evidenced. We claim the collapse mechanics and the receipt trail; we do not certify any detector's hit rate (§6).

2.3 Modeled attacker actions

The Tier-0 adversarial corpus (§5.2) operationalizes the in-flight attacker: payload field mutation, single-byte flips, prototype pollution, receipt replay, mass replay flooding, Unicode normalization collisions, homoglyph spoofing, cross-request nonce swaps, and double-execution races; plus four security games (mutation resistance, replay resistance, cross-transaction binding, serialization ambiguity). These model an adversary who can observe, mutate, reorder, and replay traffic between the agent runtime and the gateway.

2.4 Out of scope

Not addressed by this design and not claimed: semantic correctness or alignment of model output; compliance with any regulation; hardware attestation (no Nitro/TEE flow is implemented or claimed); denial of service against the gateway itself; covert channels inside authorized outputs; and the *confused deputy* within an authorized clearance — an agent that legitimately holds a floor and routes authorized data to a hostile but allowlisted destination is constrained only by routing policy, not by this paper's mechanisms.

3 Design

3.1 Agent loops as isolation levels

Table 1 summarizes the mapping that motivates the design. The point of the table is not taxonomy but a gap: nothing in the deployed ecosystem occupies the serializable row, because nothing has a validation phase. Ghost-Ark's contribution is that row.

Isolation	Agent analogue	Characteristic anomaly
Read uncommitted	API-coupled loop; mutate per step	dirty writes: pre-abort steps already leaked
Read committed	sandbox + buffered intent pool (e.g., NemoClaw-style isolated execution [13])	write skew: stale reasoning flushed onto shifted state
Serializable (OCC)	Ghost-Ark: speculate in $G(\sigma_0)$, validate, then bind	aborts under conflict; liveness cost analyzed in §4.1

Table 1: Database isolation levels and their agent-architecture analogues.

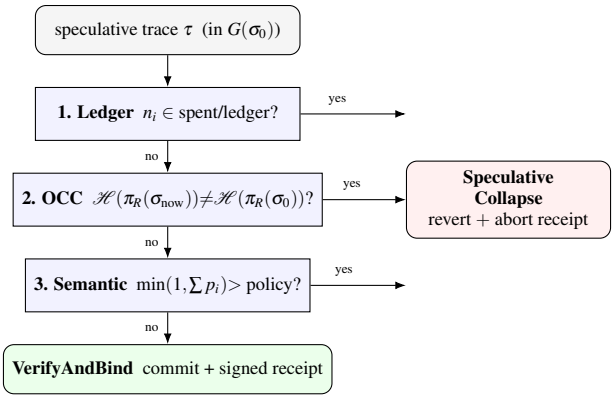

Figure 1: Three-gate validation. Every path — commit or any-gate abort — terminates in a signed receipt. Gate 2 (OCC over π_R) is specified but not yet enforced at runtime; gates 1 and 3 are implemented and, for the ledger, wired from the TLC-verified tombstone model.

3.2 Declarative Action Binding and Δ_{DE}

The untrusted runtime never transmits an action; it transmits a *commitment*. Before requesting execution, the agent runtime serializes the intended action into a deterministic normal form (DANF) and derives a commitment digest C_I. The gateway independently re-derives C_E from the exact bytes it would execute. Define the declared–executed divergence $\Delta_{\text{DE}} = |C_I \oplus C_E|$; the gateway refuses execution unless $\Delta_{\text{DE}} = 0$. This turns serialization itself into an attack surface that must be closed rather than trusted: cross-language normalization (TypeScript emitter, Rust checker) is exactly where Unicode collisions and field-injection attacks live, which is why the corpus of §5.2 attacks the encoder, not just the network. **[implemented, tested]**

3.3 The ghost replica and speculative collapse

A trajectory executes against $G(\sigma_0)$, a forked view of relevant state at transaction start; its writes accumulate in a speculative buffer that touches nothing physical. Commit-time validation either *binds* the buffer to the environment or triggers `SpeculativeCollapse`: the buffer is discarded, the replica reverts to $G(\sigma_0)$, and an abort receipt is emitted. Collapse is the mechanism that makes agent fallibility survivable: a hallucinated plan that dies in validation costs a receipt, not an environment. The collapse/commit state machine is one of the five TLC-checked models (§5.1, 529 distinct states).

3.4 The three-gate validation pipeline

Validation is a conjunction of three independently answerable questions — *is it fresh*, *is it current*, *is it within policy*:

Gate 1 — Ledger (freshness). Each transaction carries a nonce n_i. Admission requires $n_i \notin \mathscr{L}_{\text{nonce}} \wedge n_i \notin S_{\text{spent}}$, where $\mathscr{L}_{\text{nonce}}$ is the active ledger and S_{spent} a tombstone set of consumed nonces. The tombstone set is not an optimization; it is the repair of a real flaw that model checking found in our own design (§4.3). Expiry *archives* a nonce into S_{spent} rather than forgetting it, so a nonce consumed at time T is still rejected at $T + \text{TTL} + 1$. **[implemented, tested] [TLC-checked, bounded model]**

Gate 2 — OCC (currency). Admission requires $\mathscr{H}(\pi_R(\sigma_{\text{now}})) = \mathscr{H}(\pi_R(\sigma_0))$: the projection of the environment onto the trajectory's declared read-set must hash identically at commit as at fork. This is backward validation in the sense of Kung–Robinson [1], with the read-set standing in for the read phase's footprint. §4.1 develops why the projection — not the global state — is validated. **[specified, not enforced at runtime]** (the receipt schema for this gate, including the serialized π_R read-set, is implemented and tested; runtime enforcement is not).

Gate 3 — Semantic (policy). Given per-step failure marginals $p_1, \ldots, p_k$ supplied by upstream assessors for the steps of τ, the gate computes the dependence-free Fréchet upper bound on the probability that *any* step introduced a violation (§4.2) and aborts if it exceeds policy. The gate aggregates marginals it is *given*; it does not itself classify content — a boundary we state precisely because eliding it is how enforcement systems come to overclaim. **[implemented, tested]**

3.5 Receipts as the unit of evidence

Every gate decision emits a receipt: a canonical JSON envelope with an exact, validated key set, deterministically serialized, with a content-derived identifier (the hash of the unsigned envelope) and a signature over the canonical bytes. Canonicalization rejects host-language non-JSON values before signing rather than coercing them; identifiers are content-derived rather than random so that identity cannot be decoupled from payload; ordering lives in per-session monotonic sequence numbers, with wall-clock timestamps recorded as evidence, never used as an ordering primitive. Receipts verify under independent implementations (Node and Python verifiers, exercised by differential tests). Signing is HMAC in local development and KMS-based in the intended AWS mode, with verification-critical key references restricted to

immutable key ARNs. A signature proves signing authorization over the payload — it does not prove the payload's semantic truth. **[implemented, tested]**

3.6 Custody and the egress boundary

A-CC (§2.1) is answered at the custody layer. Tool traffic exits only through a loopback sidecar that performs the external request, digests the raw response bytes, injects a request nonce that the response must echo, and returns a signed envelope — making *gateway-recorded* a provenance class that fabricated in-context text cannot satisfy. Provenance admission is meet-based on a lattice, so data cannot launder trust by transiting the agent's context. A byte-level reconciler compares *decoded* tool bytes against asserted content (decode-before-compare), closing the encoding-mismatch corner of FSA attacks. The empirical shape of this boundary in end-to-end harness runs: egress that would bypass recording *fails closed* — a compromise that bypasses the gateway does not produce a side-effect, it produces a refusal, so within the modeled boundary every effect that does occur carries a recorded envelope. **[implemented, tested]** (single-node; the sidecar's SSRF-class exposures — DNS rebinding, redirect handling — are relocated into a smaller trusted surface, not dissolved).

3.7 End-to-End Scenario: Stopping Trust Laundering

Consider an adaptive agentic worm that uses indirect prompt injection to commandeer an agent, instructing it to forward sensitive customer records to an attacker-controlled endpoint.

1. **Execution in $G(\sigma_0)$:** The agent parses the poisoned record and decides to call `HTTP_POST`. Ghost-Ark's loopback sidecar intercepts the call, treating it as a speculative write.
2. **Declarative Binding (Δ_{DE}):** The agent runtime normalizes the action into a deterministic JSON payload.
3. **Custody Evaluation:** The payload claims to be a legitimate summary of customer data. However, the byte-level reconciler detects that the destination URL was *agent-asserted* (hallucinated from the prompt injection) rather than *gateway-recorded* (passed through an authorized retrieval tool).
4. **Speculative Collapse:** The transaction fails custody evaluation (§3.6): the floor demanded by the destination-write policy exceeds agent-asserted provenance, so the A-CC attempt dies as an *unsatisfiable floor* — a deterministic lattice check, not a semantic judgment (the semantic gate of §4.2 is a separate, later check over supplied marginals).
5. **Evidence Generation:** The gateway discards the speculative buffer; the network request never executes. It emits a canonical JSON receipt containing the exact payload, a `COLLAPSE` status, the unsatisfied floor, and a signature over the canonical bytes. The attempt is contained at the boundary and evidenced in the receipt ledger either way.

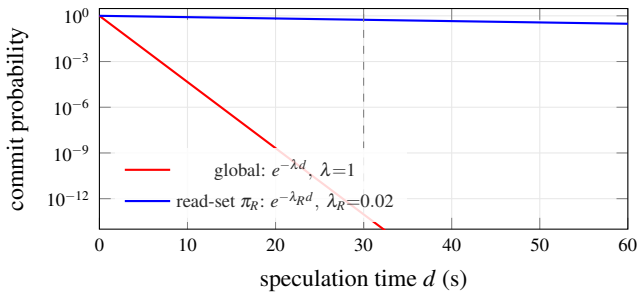

Figure 2: The starvation trap (§4.1). Global-state validation decays as $e^{-\lambda d}$ — perpetual abort ($\approx 10^{-13}$ at d=30 s, dashed line, for λ=1 background write/s). Validating only the read-set projection π_R, whose write rate $\lambda_R \ll \lambda$, keeps the commit probability near 1. The curves are the analytic Poisson model, not measurements; read-set faithfulness is the stated cost.

4 Formal Foundations

4.1 The starvation trap and the read-set projection

The naive OCC gate validates the world: $\mathscr{H}(\sigma_{\mathrm{now}}) = \mathscr{H}(\sigma_0)$. In any live environment this is a liveness catastrophe. Let environmental writes unrelated to the trajectory arrive as a Poisson process with rate λ, and let a trajectory speculate for time d. The probability that *global* validation succeeds is $e^{-\lambda d}$: for an agent that reasons for 30 s in an environment averaging one background write per second, the commit probability is $e^{-30} \approx 10^{-13}$ — perpetual abort. Worse, the abort-retry loop is self-defeating: retries lengthen exposure, and exposure breeds conflict. This is the **starvation trap**: global-state validation converts safety into a denial of liveness.

The repair is the projection operator π_R, which restricts validation to the trajectory's declared read-set — the tuples actually consulted while speculating. Validation becomes $\mathscr{H}(\pi_R(\sigma_{\mathrm{now}})) = \mathscr{H}(\pi_R(\sigma_0))$, and the abort probability depends only on the write rate *into the read-set*, $e^{-\lambda_R d}$ with $\lambda_R \ll \lambda$. The cost is stated, not hidden: π_R detects exactly read–write conflicts on declared dependencies. Predicate-shaped reads ("all alarms currently firing") must materialize their result set into the projection or the gate inherits the phantom problem [5]; an unfaithfully narrow π_R weakens conflict detection in proportion to what it omits. Serializability under π_R is therefore conditional on read-set faithfulness — a property of instrumentation, not of the gate (Figure 2). **[specified, not enforced at runtime]**

4.2 Fréchet bounds on cumulative trajectory failure

Per-step screening misses trajectory-level violations: no single action a_i crosses policy, but the sequence does. Model step i's violation as an event F_i with marginal $p_i = \mathbb{P}(F_i)$, supplied by upstream assessors. The deployment-relevant quantity is $\mathbb{P}(F_{\mathrm{any}}) = \mathbb{P}(\bigcup_i F_i)$ — and the joint dependence structure of the F_i is unknown. Assuming independence is not

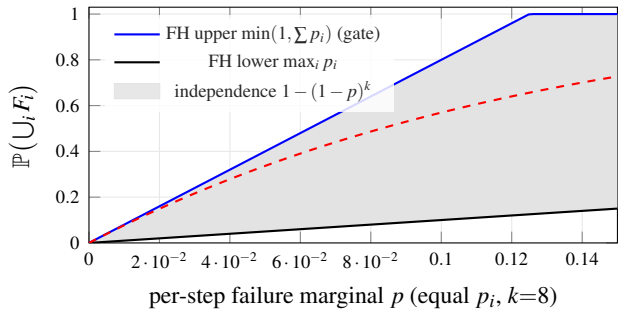

Figure 3: The dependence-free Fréchet envelope for union failure over k=8 steps, equal marginals (§4.2). The semantic gate triggers on the upper bound, which holds under *every* dependence structure. An independence assumption (dashed) lies strictly inside the envelope: it would clear trajectories the worst case flags, underreporting risk precisely where correlated guardrail failure lives. Analytic bounds, not measurements.

conservative here; it is the precise mistake our research program exists to measure, because guardrail failures *correlate*: the same poisoned context that defeats the step-3 assessment also defeats step 5's. The Fréchet bounds [19, 20] are the sharp dependence-free envelope:

$$\max_i p_i \;\le\; \mathbb{P}\Big(\bigcup_{i=1}^{k} F_i\Big) \;\le\; \min\Big(1,\, \sum_{i=1}^{k} p_i\Big), \qquad (1)$$

with the dual intersection envelope $\max\big(0, \sum_i p_i - (k-1)\big) \le \mathbb{P}(\bigcap_i F_i) \le \min_i p_i$. The semantic gate implements the upper bound of (1) as its trigger: it aborts when $\min(1, \sum_i p_i)$ exceeds the policy threshold. This is the unique choice that remains valid under *every* dependence structure, including the adversarially correlated one (Figure 3).

Two consequences are worth stating plainly. First, the bound is conservative by construction, and it saturates: as k grows, $\sum_i p_i \to 1$ even for well-behaved marginals, and the gate tends fail-closed — the starvation trade of §4.1 reappearing at the semantic layer. Long-horizon deployments must either scale the policy threshold with k or invest in calibrated marginals; the gate makes that trade explicit rather than hiding it in an independence assumption. Second, the gate's output is only as meaningful as its inputs: it computes a bound over supplied marginals and does not manufacture information about content. Equation (1)'s implementation (`evaluateSemanticGate`) is unit-tested against hand-computed bounds. **[implemented, tested]**

4.3 Model checking with mutants, and the bug it caught

Five TLA+ models cover the enforcement core: the provenance lattice's admission monotonicity, the speculative collapse state machine, the transport boundary (where the byte reconciler is shown to be load-bearing rather than a parser accident), the nonce ledger, and the execution boundary. Model checking that can only pass is theater, so every claimed invariant is paired with a *mutant* model seeded with the targeted flaw; the harness requires baselines to verify clean *and* mutants to reproduce their violations, and it records raw TLC logs as committed artifacts.

The discipline caught us. The original nonce-ledger baseline violated its own `NoReplays` invariant due to a subtle interaction between garbage collection (GC) and state expiration. TLC produced the following counterexample trace:

1. **Speculate:** Agent submits transaction with `nonce = X`.
2. **Commit:** Gateway admits `X`, adding it to $\mathscr{L}_{\text{nonce}}$.
3. **GC Tick:** `TTL` expires. `GarbageCollect` removes `X` from $\mathscr{L}_{\text{nonce}}$ to bound memory.
4. **Replay:** Attacker replays the original transaction. Because `X` is no longer in $\mathscr{L}_{\text{nonce}}$, the gateway treats it as a fresh request and admits it again.

The flaw was a true positive in the shipped design, not an artifact of the model. The repair introduced the spent-tombstone set (S_{spent}): GC now *archives* rather than *forgets*. Admission requires $n_i \notin \mathscr{L}_{\text{nonce}} \wedge n_i \notin S_{\text{spent}}$. We propagated this fix spec-first into the Rust implementation. The gateway initially retained TTL-eviction semantics — diverging from the verified model by exactly the refuted behavior — and was then brought into conformance: `cleanup_expired()` archives into a spent set, and a nonce consumed at T is rejected at $T+3601$. One bounded caveat survives and is stated in §6: the in-process tombstone set is capacity-bounded, and pruning at capacity reopens a theoretical replay window for sufficiently old nonces.

5 Evaluation

Setup disclosure. All measurements in this section are from recorded runs at repository HEAD on 2026-07-16: Apple M1 (8 GB RAM), macOS (Darwin 24.5.0, arm64), Node v22.22.3, single process, no network, no KMS — the in-process TypeScript enforcement path under the modeled attacker. TLC runs use tla2tools v1.8.0 with logs committed to the repository. Nothing here is a cloud measurement, and no live-AWS evidence is claimed. Every number maps to a re-runnable command in the artifact (§8).

5.1 Model checking

Table 2 summarizes the five baselines and their mutants. The two-sided oracle matters: a harness that can only report success cannot distinguish a verified invariant from a vacuous one. The mutant rows are the evidence that the invariants have teeth; the `NoReplays` mutant in particular reproduces the historical design flaw of §4.3 as a permanent regression witness.

5.2 Adversarial corpus

At HEAD, the nine-attack corpus (§2.3) reports every attack *detected* in-suite, and the four security games report attacker advantage 0 over 10,000 trials each (Table 3) ($\Pr[\text{win}] = 0$ across $4 \times 10{,}000$ trials; a zero observed rate over 10^4 trials

Model	Distinct states	Result
ProvenanceLattice	403,949	clean
mutant	61	violation reproduced
SpeculativeCollapse	529	clean
mutant	240	violation reproduced
TransportBoundary	64	clean
mutant	22	violation reproduced
DAB_NonceLedger	1,321	clean (incl. temporal)
mutant	232	`NoReplays` violated
DAB_ExecutionBoundary	51,106	clean

Table 2: TLC results at HEAD (tla2tools v1.8.0; recorded logs committed). Baselines must verify clean; mutants must reproduce their seeded violations. These are bounded models of the design, not proofs of the implementation.

Security game	Trials	Adversary advantage
Mutation resistance	10,000	0
Replay resistance	10,000	0
Cross-transaction binding	10,000	0
Serialization ambiguity	10,000	0

Table 3: Formal security games at HEAD (recorded run, 2026-07-16). Advantage is the fraction of trials the modeled attacker wins. In-suite results under the modeled attacker; the non-claim header of `run_all.ts` is normative.

is consistent, at 95% confidence, with a true per-trial success probability below $\approx 3 \times 10^{-4}$ [21], and says nothing about attacks outside the modeled family). The aggregate `global_advantage` $= 0$ with all suites passing.

The scope qualifier is not fine print; it is the result's semantics. These are *in-suite detection rates under the modeled attacker* — an adversary limited to the mutation/replay/encoding/interleaving family of §2.3, implemented in-process against the TypeScript enforcement path. The result is evidence that the commitment discipline closes the modeled channels; it is not a statement about unmodeled attackers, the Rust TCB under deployment conditions, or semantic-layer deception.

On indirect prompt injection: the published InjecAgent benchmark [10] reports attack success rates up to 47% for a ReAct-prompted GPT-4 agent in its enhanced setting. Under Ghost-Ark's custody model the corresponding trust-laundering move is *structurally* unsatisfiable — injected in-context text carries no gateway envelope, so it cannot meet floors above agent-asserted provenance, and our in-repository scenario tests confirm the admission rule blocks the modeled laundering pattern. We deliberately do not report this as "ASR reduced from 47% to 0%": we have not re-run the InjecAgent dataset against a live model behind Ghost-Ark, and the published 47% baseline was measured under different conditions. The honest statement is architectural non-satisfiability under the modeled floor semantics, evidenced by unit and scenario tests.

Stage	p50	p95	p99	mean	ops/s
Baseline dispatch	0.46	0.50	0.58	0.50	2,019,173
DANF commit (C_I)	3.38	3.75	13.13	4.03	248,275
Gateway verify (C_E)	0.92	1.08	5.17	1.15	870,902
End-to-end	5.5	6.3	24.6	7.1	140,941

Table 4: Enforcement-path latency in *microseconds* (10,000 iterations, 1,024-byte payloads, in-process, Apple M1).

5.3 Performance

Table 4 reports the enforcement path's cost. The end-to-end commitment–verification cycle costs 5.5 μs at the median (7.1 μs mean, 24.6 μs p99) and sustains ≈140,941 ops/s single-threaded — an absolute added cost of $\approx 6.6\,\mu$s mean over baseline dispatch.

We decline to headline the harness's own relative metric, and the reason is instructive. Computed as $(\text{DAB} - \text{baseline})/\text{baseline}$, the run reports an overhead of ≈ 1,333% — which sounds disqualifying until one notices the denominator is a 0.5 μs no-op dispatch. Relative overhead against a near-zero baseline is a number without decision content: against any realistic tool call (a network round-trip at 10^4–10^6 μs, a KMS signing call at 10^3–10^4 μs), an added 6.6 μs of in-process computation is far below measurement noise. The deployment-relevant statement is the absolute one, and the honest caveat is its converse: these numbers exclude KMS signing latency and all network costs, which will dominate in the intended cloud mode and which we have not measured (§6). "Fast" is not a property we claim; "microseconds of in-process arithmetic, unmeasured I/O excluded" is.

Concurrent TCB throughput, measured. Table 4 times the *TypeScript* commitment cycle — canonical serialization plus SHA-256 digest and comparison; it performs no signature, and its ops/s column is single-thread inverse latency, not a load test. We therefore also measure the Rust TCB directly under concurrency: 64 threads driving the lock-free sharded replay ledger, each admitted transaction paying for a real ed25519 signature, sustain 275,415 admissions+signatures/s within ledger capacity (96,000/96,000 admitted), and the fail-closed rejection path at capacity sustains $\approx 10.1 \times 10^6$ rejections/s, on the same M1 (recorded run committed to the artifact). The harness is two-sided: every in-capacity op must admit and every at-capacity op must refuse, or the run exits non-zero. This is the in-process ceiling of the TCB itself — it excludes the Unix-socket transport and all network I/O, and it is not a deployed-load claim (§6).

5.4 A disclosed harness bug

An earlier revision of this benchmark scored two suites backwards: the replay game counted a *detected* replay as an attacker win (reporting advantage ≈ 0.998 where correct accounting yields 0), and one mutation probe inverted its detection predicate, yielding an aggregate "advantage" of ≈ 0.25 that contradicted the design's own ledger semantics. The error was in the harness's accounting, not the enforcement path

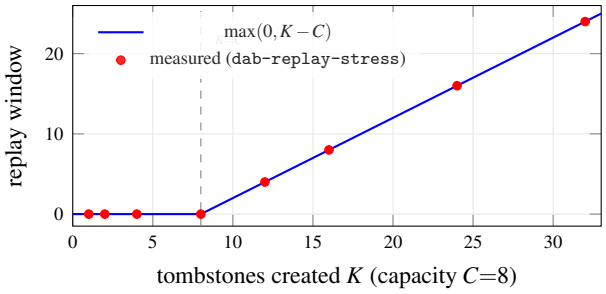

Figure 4: The bounded replay window, measured (§5.5). Below capacity the window is empty; above it, every additional tombstone is one more replayable nonce. Measured points fall exactly on $\max(0, K-C)$ (2). This is a measurement of the mechanism's own stated limitation, not of its strength.

— but for a period the repository's own artifact report published the red result rather than patching around it, and the fix landed as an ordinary reviewed commit. We disclose this for the same reason we ship mutant models: an evaluation pipeline that cannot be caught being wrong cannot be trusted when it says it is right.

5.5 The replay window, measured

The ledger gate's one honest weakness (§6, item 5) is that its in-process tombstone set is capacity-bounded: at capacity, pruning reopens a replay window for old nonces. Most papers state such a caveat and stop. We measure it. Driving the real `ReplayLedger` at capacity C after creating K tombstones (TTL= 0 for a deterministic boundary) and counting how many consumed nonces become replayable (`!exists`, exactly the condition under which `consume` would re-accept) yields, across capacities from 8 to 100 and K up to 1000 (recorded in `RECORDED_REPLAY_WINDOW.txt`), an exact law:

$$\text{replay window} \;=\; \max\big(0,\, K-C\big). \qquad (2)$$

Figure 4 shows the knee at K=C: below capacity the window is empty (every tombstone is retained), above it the window grows one-for-one. The measurement also surfaces a *second* finding the caveat alone hides: because the in-process set is a `HashSet`, the pruned membership is implementation-arbitrary, not age-ordered — so a durable production store must use time-ordered eviction, not merely more capacity. The bound is now a number an operator can size against, not a hand-wave: with the default C=500,000 the window stays empty until more than half a million distinct nonces outlive their TTL between compactions.

5.6 Deployment Sketch: Ghost-Ark on AWS

This subsection is a *design target*, not an evaluated system: no live-cloud measurement appears in this paper, and items 3 and 6 of §6 govern everything below. With that boundary stated, the architecture maps onto standard cloud infrastructure as follows. The untrusted agent runtime (e.g., a ReAct loop calling a hosted model API) would run in an isolated container, with the Ghost-Ark gateway as an out-of-process sidecar; the speculative buffer $G(\sigma_0)$ stays local to the gateway. At commit time the ledger gate's tombstone set would move from the in-process bounded structure to a durable KV store using conditional writes (the posture already recommended for the capacity caveat of §6, item 5), and the OCC gate — still **[specified, not enforced at runtime]** — would validate π_R digests against the same store. Receipt signing in the intended cloud mode uses KMS asymmetric keys referenced by immutable key ARNs, per the repository's signing boundary. Cloud I/O (network round-trips, KMS calls) would dominate the $\approx 6.6\,\mu$s in-process arithmetic of §5.3 by orders of magnitude; the validation *structure* — the same three-gate conjunction, the same receipt semantics — is what carries over, while cloud-mode latency, availability, and failure behavior remain unmeasured claims we do not make.

6 Limitations and Non-Claims

Ghost-Ark verifies what was *recorded, signed, policy-bounded, and replayable under its verifier rules*. Each item below is a boundary we consider part of the contribution, not an escape hatch.

1. **No semantic safety.** Nothing here certifies that agent output is true, aligned, or safe. The semantic gate bounds a function of supplied marginals; garbage marginals yield a well-computed bound over garbage.
2. **Bounded models, not verified implementations.** TLC results are exhaustive only within stated bounds and hold for the models; conformance of the Rust/TypeScript implementations is tested, not proved.
3. **OCC gate unimplemented at runtime.** π_R validation exists as specification and tested receipt schema; no runtime enforces it yet. Serializability claims are therefore design-level, and conditional on read-set faithfulness even then.
4. **Modeled attacker only.** The 0-advantage result covers the nine-attack/four-game family, in-process. No claim covers unmodeled attacks, the deployed TCB, or availability.
5. **Bounded tombstone capacity.** The in-process spent set prunes at 500,000 entries (env-tunable); a nonce older than both TTL and tombstone capacity has a replay window — *measured* to be exactly $\max(0, K-C)$ for K tombstones at capacity C, with implementation-arbitrary membership (§5.5, Figure 4). Durable external stores with time-ordered eviction (e.g., conditional writes against a database) are the intended production posture and are not implemented here.
6. **No live-cloud evidence.** KMS-mode signing, Bedrock-path integration, and all AWS-live behavior are design targets; no live-AWS measurement appears in this paper.
7. **No attestation.** Signing proves authorization over payload bytes; it does not prove runtime integrity, and no enclave/PCR-bound flow is claimed.
8. **Confused deputy within clearance** (§2.4) is mitigated by routing policy at best; the lattice checks floors, not intent.

9. **Single node.** All ledgers are single-writer; multi-node consensus over ledger state is future work, not a property of this system.
10. **No detector efficacy claims.** Where upstream assessors supply p_i, their hit rates are theirs; this paper certifies none of them.

7 Related Work

Transactions and isolation. The validation-phase structure is Kung–Robinson OCC [1]; the speculative-buffer discipline descends from STM [2, 3]; serializability theory and the anomaly vocabulary are classical [4, 5, 6]. Ghost-Ark's difference is the transaction participant: a stochastic planner whose failure modes are adversarial and correlated, not merely concurrent.

Information flow. The provenance lattice is in the Denning lineage [7] with language-based descendants such as JIF [8]; the custody twist is that labels are *cryptographically evidenced* classes (gateway-recorded vs. agent-asserted) rather than compiler-tracked types.

Agent security. Indirect prompt injection is systematized in [9]; InjecAgent [10] benchmarks it for tool-integrated agents; ReAct [11] is the canonical coupled loop this paper's Table 1 places at read uncommitted. Sandbox isolation is complementary at two layers — infrastructure microVMs (Firecracker [12]) and agent-focused confinement stacks such as NVIDIA NemoClaw, which runs the agent under Landlock/seccomp/network-namespace isolation with policy-gated egress [13]. Both bound the blast radius of a *process*; Ghost-Ark bounds the *admissibility of its effects* and evidences every decision — Table 1's read-committed row is exactly the sandbox posture, strengthened here with a validation phase.

Transparency. Receipt chains and independent replay are in the spirit of Certificate Transparency [14]: shrink the trusted statement from "the system behaved" to "the log is append-only and the artifacts verify".

Formal methods in systems. The TLA+/TLC workflow [15, 16] and its industrial use at scale [17] inform the harness; the mutant-oracle discipline is mutation testing [18] applied to specifications rather than code.

8 Artifact Availability

The repository ships a reviewer container (Ubuntu 22.04 with Node 22, a JDK, pinned tla2tools, and TeX Live) and a one-command reproduction: `make reproduce` runs build → claim-language gate → TLC (baselines and mutants) → unit/integration (640 tests, 97 files at HEAD) → adversarial corpus → benchmarks, and writes a machine-readable pass/fail roll-up whose gating status is the exit code. A claim-to-command map (`README-AE.md`) binds every number in this paper to the script that regenerates it. The repository also runs a forbidden-claims scanner over its own documentation (588 scannable files at HEAD, zero violations) — the same discipline this section is subject to.

9 Conclusion

The agent-security debate keeps asking how to make the model trustworthy. Databases stopped asking the analogous question about transactions fifty years ago: assume the client is wrong, isolate its speculation, validate at commit, and log everything. Ghost-Ark is that posture applied to nondeterministic compute — a ledger gate that model checking humbled and then hardened, an OCC discipline honest about its liveness trade, a semantic bound honest about its inputs, and a receipt for every decision either way. What it refuses to claim is part of the design: a skeptical reviewer should be able to distrust the authors, the README, and the model, and still replay the digest, verify the signature, map each claim to its evidence, and reproduce the failure boundary.

Acknowledgments

The author utilized an AI assistant to help format, refine, and draft the prose of this manuscript; however, the architectural design, formal modeling, experimental execution, and final conclusions are the sole work of the author, who takes full responsibility for the contents of this paper.

A Artifact Appendix

A.1 Abstract

The artifact is the complete repository: the TypeScript enforcement runtime and receipt schema, the Rust gateway sources, five TLA+ models with five mutants and their recorded TLC logs, the nine-attack adversarial corpus and four security games, the performance harness, the claim-language scanner that gates the repository's own documentation (including this manuscript's `.tex` source), and a one-command reproduction pipeline that writes a machine-readable pass/fail roll-up. It reproduces every number in Tables 2–4.

A.2 Checklist

- **Program:** Node.js 22 (TypeScript via native type-stripping), JDK $\geq$ 11 for TLC (pinned `tla2tools.jar`), optionally Rust stable for the gateway tests.
- **Run-time environment:** Linux x86_64 or macOS arm64; or the provided Ubuntu 22.04 reviewer container (zero host setup beyond Docker).
- **Hardware:** any commodity machine; the paper's latency numbers are from an Apple M1 (8 GB) and reproduce within machine variance.

- **Metrics:** attacker advantage, detection flags, TLC distinct-state counts, latency percentiles, test counts, claim-gate status.
- **Expected wall-clock:** proofs ∼15–25 s; corpus + games ∼30–60 s; full unit suite ∼2.5–3.5 min; `make reproduce` end-to-end ∼4–5 min (46 s on the reference machine with a warm dependency cache).
- **Public availability:** the repository is public; an immutable archived snapshot (e.g., Zenodo DOI) is the intended citable artifact.

A.3 Claims mapped to commands

The authoritative map is `README-AE.md` at the repository root; the digest:

- Table 2 (models clean, mutants violate, state counts): `make proof` → `artifacts/proofs/proofs_summary.json`. Exact match.
- Table 3 and the nine-attack corpus: `node --experimental-strip-types dab/bench/run_all.ts --trials 10000` → `global_advantage: 0`, `all_passed: true`. Exact match.
- Table 4: same command (the `performance` block) or `make benchmark`. Microsecond scale reproduces; exact values are machine-dependent.
- Test counts (640/97): `make unit`. Exact.
- Claim gate (588 files, 0 violations): `npm run scan:claims`. Exact.
- Semantic-gate bound (§4.2): `npx vitest run tests/unit/receipt-schema/semanticAuditReceipt.test.ts`. Exact.
- Full roll-up: `make reproduce` (container: `docker compose -f docker-compose.reviewer.yml run --rm reviewer make reproduce`); exit code 0 iff every gating stage passed.

A.4 The gateway↔verifier round-trip and its enforcement

The Rust gateway and the independent verifier agree on a real ed25519 signature over a domain-separated canonical message including `policy_digest`. A recorded, reproducible round-trip (`dab/roundtrip/`) shows a gateway-emitted receipt verifying, and tamper / mutation / wrong-key variants each rejected with the specific error; the same round-trip is demonstrated on Kubernetes (`dab/k8s/`: a gateway init-container emits the receipt, a separate verifier container accepts it in-cluster). A full socket-transport E2E (`dab/roundtrip/run_socket_e2e.sh`) drives the running gateway over `/ipc/dab.sock` with a real agent client and shows the *TLC-verified spent-tombstone ledger*, now wired into the gateway binary, rejecting a replayed nonce (`REPLAY_REJECTED`) while certifying the honest request. The gateway and verifier crates are `cargo clippy -D warnings` clean. Scope: a local DEV ed25519 key (not KMS/HSM/TPM/Nitro); the bounded tombstone-capacity caveat (§6, item 5) is unchanged.

A.5 What the artifact deliberately does not show

No live-cloud behavior (KMS-mode signing, cloud latency); nothing semantic. The reproduction pipeline publishes its own failures: at earlier commits it exited non-zero on real blockers — including a period when the benchmark's inverted accounting (§5.4) kept the artifact red — and the RESOLVED entries retain that history. An evaluator who wants to test the fail-closed behavior can reintroduce any forbidden phrase into a scanned file and watch the claim stage refuse.

References

[1] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2), 1981.

[2] N. Shavit and D. Touitou. Software transactional memory. In *PODC*, 1995.

[3] M. Herlihy and J. E. B. Moss. Transactional memory: architectural support for lock-free data structures. In *ISCA*, 1993.

[4] C. H. Papadimitriou. The serializability of concurrent database updates. *J. ACM*, 26(4), 1979.

[5] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. In *SIGMOD*, 1995.

[6] P. A. Bernstein and N. Goodman. Concurrency control in distributed database systems. *ACM Comput. Surv.*, 13(2), 1981.

[7] D. E. Denning. A lattice model of secure information flow. *Commun. ACM*, 19(5), 1976.

[8] A. C. Myers and B. Liskov. A decentralized model for information flow control. In *SOSP*, 1997.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec*, 2023.

[10] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. InjecAgent: benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of ACL*, 2024.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: synergizing reasoning and acting in language models. In *ICLR*, 2023.

[12] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In *NSDI*, 2020.

[13] NVIDIA. NemoClaw architecture overview. `https://docs.nvidia.com/nemoclaw/latest/`, 2026. Accessed 2026-07-16.

[14] B. Laurie. Certificate transparency. *Commun. ACM*, 57(10), 2014.

[15] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[16] Y. Yu, P. Manolios, and L. Lamport. Model checking TLA+ specifications. In *CHARME*, 1999.

[17] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services uses formal methods. *Commun. ACM*, 58(4), 2015.

[18] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer*, 11(4), 1978.

[19] R. B. Nelsen. *An Introduction to Copulas*, 2nd ed. Springer, 2006.

[20] G. Boole. *An Investigation of the Laws of Thought*. Walton and Maberly, 1854.

[21] J. A. Hanley and A. Lippman-Hand. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA*, 249(13), 1983.